

Dossier de Conception Technique

Mini Garage - Application de gestion de garage automobile

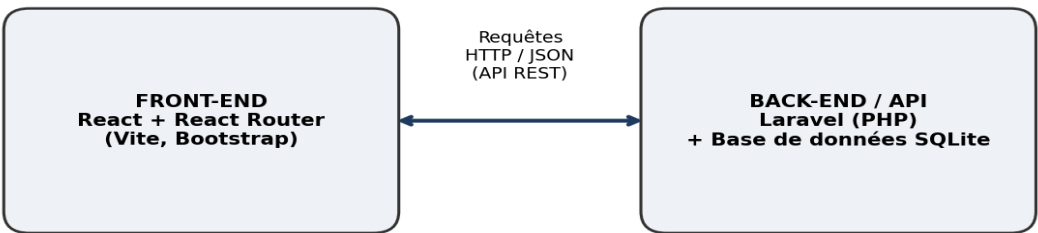
1. Introduction

L'objectif de « Mini Garage » est de fournir au personnel d'un garage automobile une application web pour gérer les véhicules pris en charge, les réparations effectuées et les techniciens qui y interviennent.

Ce document détaille les choix techniques et structurels qui permettent de réaliser les fonctionnalités définies dans le Cahier des Charges (DSF). L'application est développée avec React pour le frontend et Laravel pour le backend (API).

2. Architecture Générale

L'application suit une architecture client-serveur découplée : une application monopage (SPA) React côté frontend communique avec une API RESTful Laravel côté backend, via des requêtes HTTP au format JSON.



Architecture générale - Frontend React / API Laravel

Base de données : SQLite en environnement de développement (fichier unique, simple à mettre en place), migrable vers MySQL ou PostgreSQL sans changement de code métier.

3. Modèle de Données (Structure de la Base de Données)

Trois tables principales structurent l'application, reliées entre elles comme suit : un véhicule possède plusieurs réparations, et une réparation peut être assignée à plusieurs techniciens (relation plusieurs-à-plusieurs via une table de liaison).

Table : vehicules

Colonne	Type	Contrainte
id	INT	Clé primaire, auto-incrément
immatriculation	VARCHAR(20)	Obligatoire, unique
marque	VARCHAR(100)	Obligatoire
modele	VARCHAR(100)	Obligatoire

Colonne	Type	Contrainte
couleur	VARCHAR(50)	Obligatoire
annee	INT	Obligatoire
kilometrage	INT	Obligatoire
carrosserie	VARCHAR(100)	Obligatoire
energie	VARCHAR(50)	Obligatoire
boite	VARCHAR(50)	Obligatoire
created_at / updated_at	DATETIME	Générés automatiquement

Table : reparations

Colonne	Type	Contrainte
id	INT	Clé primaire, auto-incrément
vehicule_id	INT	Clé étrangère → vehicules.id, suppression en cascade
date	DATE	Obligatoire
duree_main_oeuvre	INT	Obligatoire (en heures)
objet_reparation	VARCHAR(255)	Obligatoire
created_at / updated_at	DATETIME	Générés automatiquement

Table : techniciens

Colonne	Type	Contrainte
id	INT	Clé primaire, auto-incrément
nom	VARCHAR(100)	Obligatoire
prenom	VARCHAR(100)	Obligatoire
specialite	VARCHAR(100)	Obligatoire
created_at / updated_at	DATETIME	Générés automatiquement

Table pivot : reparation_technicien

Colonne	Type	Contrainte
id	INT	Clé primaire, auto-incrément
reparation_id	INT	Clé étrangère → reparations.id, cascade
technicien_id	INT	Clé étrangère → techniciens.id, cascade

Relations

- Vehicule hasMany Reparation / Reparation belongsTo Vehicule.

- Reparation belongsToMany Technicien (via reparation_technicien) / Technicien belongsToMany Reparation.

Ce modèle correspond directement aux migrations Laravel déjà en place dans le projet.

4. Conception de l'API Backend (Laravel)

L'API expose trois ressources REST principales, générées via **Route::apiResource**, ce qui fournit automatiquement les 5 routes CRUD standard pour chacune.

Endpoints - Véhicules

Méthode	URL	Description
GET	/api/vehicules	Liste des véhicules (paramètre ?search= pour filtrer par marque ou immatriculation)
GET	/api/vehicules/{id}	Détail d'un véhicule, avec ses réparations et leurs techniciens
POST	/api/vehicules	Créer un véhicule
PUT	/api/vehicules/{id}	Modifier un véhicule
DELETE	/api/vehicules/{id}	Supprimer un véhicule

Endpoints - Techniciens

Méthode	URL	Description
GET	/api/techniciens	Liste des techniciens
GET	/api/techniciens/{id}	Détail d'un technicien avec ses réparations
POST	/api/techniciens	Créer un technicien
PUT	/api/techniciens/{id}	Modifier un technicien
DELETE	/api/techniciens/{id}	Supprimer un technicien

Endpoints - Réparations

Méthode	URL	Description
GET	/api/reparations	Liste des réparations, avec véhicule et techniciens associés
GET	/api/reparations/{id}	Détail d'une réparation
POST	/api/reparations	Créer une réparation (+ association techniciens)
PUT	/api/reparations/{id}	Modifier une réparation (+ synchronisation techniciens)
DELETE	/api/reparations/{id}	Supprimer une réparation

Depuis l'ajout de l'authentification (voir ci-dessous), tous ces endpoints nécessitent désormais un token Sanctum valide, à l'exception de /api/login. Toutes les entrées sont par ailleurs validées côté serveur via la méthode validate() de Laravel avant tout enregistrement.

Endpoints - Authentification

Toutes les routes ci-dessus (véhicules, techniciens, réparations) sont protégées par le middleware auth:sanctum. Seules les routes suivantes sont publiques ou dédiées à la connexion :

Méthode	URL	Description
/login	Connexion administrateur (accessible sans authentification)	Page de connexion
POST	/api/login	Authentifie l'administrateur (email + mot de passe) et renvoie un token Sanctum
POST	/api/logout	Révoque le token courant (protégé, nécessite d'être connecté)
GET	/api/me	Renvoie l'utilisateur actuellement authentifié (protégé)

5. Structure du Frontend (React)

Pages / routes

Route	Page
/	Accueil
/vehicules	Liste des véhicules (recherche, vue tableau/cartes)
/vehicules/:id	Détail d'un véhicule
/vehicules/create	Création d'un véhicule
/vehicules/edit/:id	Modification d'un véhicule
/techniciens	Liste des techniciens
/techniciens/:id	Détail d'un technicien
/techniciens/create	Création d'un technicien
/techniciens/edit/:id	Modification d'un technicien
/reparations	Liste des réparations
/reparations/:id	Détail d'une réparation
/reparations/create	Création d'une réparation
/reparations/edit/:id	Modification d'une réparation

Composants principaux

- Navbar : barre de navigation persistante, avec mise en évidence de l'onglet actif via `useLocation`.
- AppRoute : composant central de routage (`react-router-dom`) qui déclare l'ensemble des routes ci-dessus.
- Pages « Liste » (Vehicules, Techniciens, Reparations) : chargement des données via `useEffect` + `fetch`, gestion de la recherche pour les véhicules.

- Pages « Détail » : affichage d'une fiche complète et action de suppression avec confirmation.
- Pages « Formulaire » : formulaire unique réutilisé pour la création et la modification (détection du mode via le paramètre :id), avec validation des champs côté client avant envoi.

Gestion de l'état

Pas de gestion d'état global (pas de Context API ni de Redux) : chaque page gère son propre état local via useState et useEffect, ce qui est suffisant pour la taille du projet.

Navigation

Assurée par react-router-dom (Routes, Route, Link, useNavigate, useParams), avec des redirections après création/modification/suppression.

Communication avec l'API

Les appels utilisent l'API Fetch native, à l'URL définie par la variable d'environnement VITE_BACKEND_URL, ce qui permet de changer facilement d'environnement (local, démonstration) sans modifier le code.

6. Réflexions sur les Exigences Non Fonctionnelles Clés

Sécurité : l'accès à l'API est protégé par Laravel Sanctum (authentification par token) - toutes les routes métier exigent un token valide, seule /api/login est publique. Les identifiants de l'administrateur (email + mot de passe) sont configurés via des variables d'environnement (ADMIN_EMAIL / ADMIN_PASSWORD) plutôt qu'en dur dans le code, et synchronisés par le DatabaseSeeder. Toutes les données sont par ailleurs validées côté serveur par Laravel (types, champs requis, unicité de l'immatriculation, existence des identifiants liés - vehicule_id, technicien_id). L'ORM Eloquent protège nativement contre les injections SQL.

Performance : les relations sont chargées en une seule requête via with() (ex. Reparation::with(['vehicule', 'techniciens'])) plutôt qu'en plusieurs appels séparés, ce qui évite le problème des requêtes N+1. La recherche de véhicules se fait côté serveur (clause WHERE ... LIKE) plutôt que de charger tous les véhicules côté client.

Éco-responsabilité : les listes ne renvoient que les colonnes nécessaires à l'affichage, SQLite est un moteur léger adapté au volume de données du projet, et l'interface Bootstrap reste sobre (pas d'images lourdes, chargement minimal de ressources).

7. Planification Détaillée de la Réalisation

Le développement React et Laravel du cœur de l'application (modèles, migrations, contrôleurs API, routes, pages et composants React) a déjà été réalisé lors d'une itération précédente. La phase de conception documentée ici formalise a posteriori les choix effectués, et la semaine 6 sera consacrée à :

- Jours 8-9 : Revue et finalisation des pages et composants React existants (cohérence UI, gestion des erreurs API).
- Jours 10-11 : Revue des contrôleurs et validations Laravel, ajout de règles manquantes si nécessaire.
- Jour 12 : Vérification de l'intégration complète front/back (tous les scénarios CRUD).
- Jour 13 : Écriture de tests backend (PHPUnit/Pest) sur les endpoints principaux.
- Jour 14 : Optimisation, sécurité et éco-responsabilité (revue des requêtes, validations).
- Jour 15 : Dépôt GitHub, README, préparation de la démonstration.

8. Points à Clarifier / Décisions à Prendre Pendant le Développement

- Faut-il ajouter une pagination sur la liste des véhicules/réparations si le volume de données augmente ?
- Faut-il exposer un endpoint de suppression groupée des techniciens d'une réparation, ou conserver la synchronisation complète (sync) actuelle ?

9. Déploiement

L'application est déployée publiquement pour la démonstration :

Composant	Plateforme	Lien
Front-end (React)	Netlify	https://mini-garage.netlify.app/
Back-end (API Laravel)	Render (Docker)	https://mini-garage-api-o7m5.onrender.com/
Base de données	Render PostgreSQL	(accès interne, non public)
Présentation vidéo	Google Drive	Lien vers la vidéo

Le back-end tourne dans un conteneur Docker (PHP 8.4 + Apache) sur Render ; les migrations et le seeding (données de démonstration + compte administrateur) s'exécutent automatiquement à chaque démarrage. Le front-end est buildé avec Vite et déployé statiquement sur Netlify, configuré pour appeler l'API via la variable d'environnement VITE_BACKEND_URL.

- Quel niveau de tests automatisés (unitaires vs fonctionnels) est réaliste dans le temps restant de la semaine 6 ?